

WorkSpace and ROS Topic

Prof.: Dr. Oybek Eraliev

TA: Jasurbek Mamurov



Learning Objectives:

- Create Python publisher and subscriber nodes from scratch
- Understand one-to-many, many-to-one, and many-to-many communication
- Build custom message types
- Visualize and verify communication with ROS tools

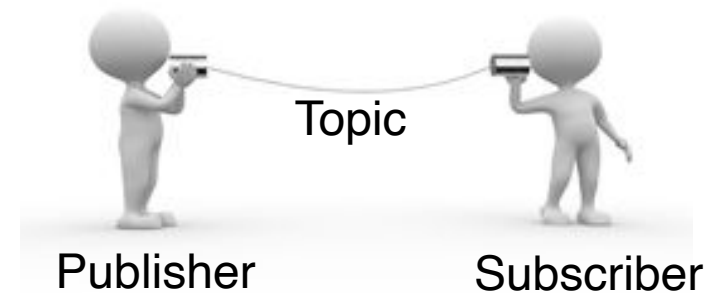
Content:

- Publisher node with Python
- Subscriber node with Python
- Message type and structure
- One-to-many message communications
- Many-to many message communications

Publisher node with Python

```
cd ~/catkin_ws/src/my_robot_pkg  
mkdir scripts  
touch scripts/simple_publisher.py  
chmod +x scripts/simple_publisher.py
```

Main library used in development is rosy
Main page: <https://wiki.ros.org/rosy>



Publisher node with Python

```
#!/usr/bin/env python3
# simple_publisher.py
import rospy
from std_msgs.msg import String

def publisher_node():
    # Initialize the node with a name
    rospy.init_node('simple_publisher', anonymous=False)

    # Create a publisher
    # Arguments: topic_name, message_type, queue_size
    pub = rospy.Publisher('/my_topic', String, queue_size=10)

    # Set the publish rate (10 Hz)
    rate = rospy.Rate(10)

    rospy.loginfo("Publisher node started!")
```

Publisher node with Python

```
while not rospy.is_shutdown():
    message = String()
    message.data = f"Hello from publisher! Time: {rospy.get_time():.2f}"

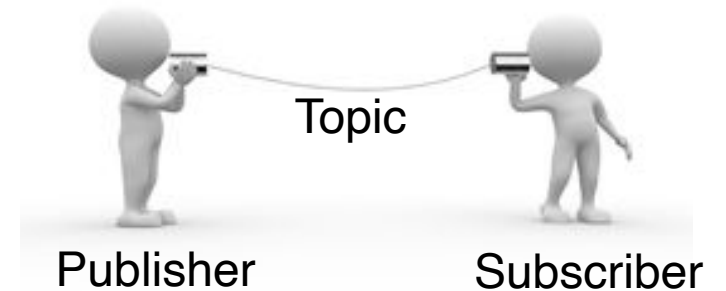
    pub.publish(message)
    rospy.loginfo(f"Published: {message.data}")

    rate.sleep() # Sleep to maintain 10 Hz rate

if __name__ == '__main__':
    try:
        publisher_node()
    except rospy.ROSInterruptException:
        pass
```

Subscriber node with Python

```
cd ~/catkin_ws/src/my_robot_pkg  
mkdir scripts  
touch scripts/simple_subscriber.py  
chmod +x scripts/simple_subscriber.py
```



Subscriber node with Python

```
#!/usr/bin/env python3
# simple_subscriber.py
import rospy
from std_msgs.msg import String

def callback(message):
    """This function is called every time a message arrives on the
    topic."""
    rospy.loginfo(f"Received: {message.data}")

def subscriber_node():
    rospy.init_node('simple_subscriber', anonymous=False)

    # Create a subscriber
    # Arguments: topic_name, message_type, callback_function
    rospy.Subscriber('/my_topic', String, callback)
```

Subscriber node with Python

```
rospy.loginfo("Subscriber node started! Waiting for messages...")

# Keep the node alive – hands control to ROS
rospy.spin()

if __name__ == '__main__':
    try:
        subscriber_node()
    except rospy.ROSInterruptException:
        pass
```

Subscriber node with Python

```
# Terminal 1  
roscore
```

```
# Terminal 2  
roslaunch my_robot_pkg simple_publisher.py
```

```
# Terminal 3  
roslaunch my_robot_pkg simple_subscriber.py
```

```
# Terminal 4 – verify  
rostopic echo /my_topic  
rqt_graph
```

Subscriber node with Python

```
# String
from std_msgs.msg import String
msg = String()
msg.data = "Hello"

# Float
from std_msgs.msg import Float32          # 32-bit float
from std_msgs.msg import Float64          # 64-bit float (more precise)
msg = Float32()
msg.data = 3.14

# Boolean
from std_msgs.msg import Bool
msg = Bool()
msg.data = True

# Integer
from std_msgs.msg import Int32             # 32-bit integer
from std_msgs.msg import Int64             # 64-bit integer
msg = Int32()
msg.data = 42
```

Content:

- ~~Publisher node with Python~~
- ~~Subscriber node with Python~~
- Message type and structure
- One-to-many message communications
- Many-to many message communications

Message type and structure

```
cd ~/catkin_ws/src/my_robot_pkg
mkdir msg
touch msg/RobotStatus.msg
^^^
^^^

# RobotStatus.msg
string robot_name
float32 battery_level
bool is_moving
int32 error_code
```

Message type and structure

Update CMakeLists.txt

```
find_package(catkin REQUIRED COMPONENTS
  rospy
  std_msgs
  message_generation          # ADD THIS
)

add_message_files(
  FILES
  RobotStatus.msg            # ADD THIS BLOCK
)

generate_messages(
  DEPENDENCIES
  std_msgs                   # ADD THIS BLOCK
)

catkin_package(
  CATKIN_DEPENDS rospy std_msgs message_runtime # ADD message_runtime
)
```

Message type and structure

Update package.xml

```
<build_depend>message_generation</build_depend>  
<exec_depend>message_runtime</exec_depend>
```

```
cd ~/catkin_ws && catkin_make  
source devel/setup.bash
```

```
# Verify your message was built  
rosmmsg show my_robot_pkg/RobotStatus
```

Message type and structure

```
#!/usr/bin/env python3
# robot_status_publisher.py

import rospy
from my_robot_pkg.msg import RobotStatus

def publisher_node():
    rospy.init_node('robot_status_publisher')
    pub = rospy.Publisher('/robot_status', RobotStatus,
queue_size=10)
    rate = rospy.Rate(1) # 1 Hz

    battery = 100.0
```

Message type and structure

```
while not rospy.is_shutdown():
    msg = RobotStatus()
    msg.robot_name = "RoboBot-1"
    msg.battery_level = battery
    msg.is_moving = True
    msg.error_code = 0

    pub.publish(msg)
    rospy.loginfo(f"Status published – Battery: {battery:.1f}%")

    battery -= 1.0 # Simulate battery drain
    if battery < 0:
        battery = 100.0

    rate.sleep()

if __name__ == '__main__':
    try:
        publisher_node()
    except rospy.ROSInterruptException:
        pass
```

Message type and structure

```
#!/usr/bin/env python3
# robot_status_subscriber.py

import rospy
from my_robot_pkg.msg import RobotStatus

def callback(msg):
    rospy.loginfo("--- Robot Status Update ---")
    rospy.loginfo(f"  Robot:      {msg.robot_name}")
    rospy.loginfo(f"  Battery:    {msg.battery_level:.1f}%")
    rospy.loginfo(f"  Moving:     {msg.is_moving}")
    rospy.loginfo(f"  Error:      {msg.error_code}")

    if msg.battery_level < 20.0:
        rospy.logwarn("LOW BATTERY WARNING!")
```

Message type and structure

```
def subscriber_node():
    rospy.init_node('robot_status_subscriber')
    rospy.Subscriber('/robot_status', RobotStatus, callback)
    rospy.spin()

if __name__ == '__main__':
    try:
        subscriber_node()
    except rospy.ROSInterruptException:
        pass
```

Message type and structure

Must run after creating new file in a package

```
catkin_make
```

```
source ~/catkin_ws/devel/setup.bash
```

Content:

- ~~Publisher node with Python~~
- ~~Subscriber node with Python~~
- ~~Message type and structure~~
- One-to-many message communications
- Many-to many message communications

One-to-many message communications

```
#!/usr/bin/env python3
# sensor_publisher.py - ONE publisher broadcasting sensor data
import rospy
from std_msgs.msg import Float32

def sensor_publisher():
    rospy.init_node('sensor_publisher')
    pub = rospy.Publisher('/temperature', Float32, queue_size=10)
    rate = rospy.Rate(2)
    import random
    while not rospy.is_shutdown():
        temp = Float32()
        temp.data = round(20.0 + random.uniform(-2, 5), 2)
        pub.publish(temp)
        rate.sleep()

if __name__ == '__main__':
    try:
        sensor_publisher()
    except rospy.ROSInterruptException:
        pass
```

One-to-many message communications

```
#!/usr/bin/env python3
# logger_subscriber.py – Subscriber 1: logs data to console

import rospy
from std_msgs.msg import Float32

def callback(msg):
    rospy.loginfo(f"[LOGGER] Temperature received: {msg.data}°C")

rospy.init_node('logger_node')
rospy.Subscriber('/temperature', Float32, callback)
rospy.spin()
```

One-to-many message communications

```
#!/usr/bin/env python3
# alert_subscriber.py – Subscriber 2: triggers alert if too hot

import rospy
from std_msgs.msg import Float32

def callback(msg):
    if msg.data > 23.0:
        rospy.logwarn(f"[ALERT] High temperature detected: {msg.data}°C!")

rospy.init_node('alert_node')
rospy.Subscriber('/temperature', Float32, callback)
rospy.spin()
```

Content:

- ~~Publisher node with Python~~
- ~~Subscriber node with Python~~
- ~~Message type and structure~~
- ~~One-to-many message communications~~
- Many-to many message communications

Many-to-one message communications

```
#!/usr/bin/env python3
# robot_publisher.py – Run this twice with different robot names via arg

import rospy, sys
from std_msgs.msg import String

robot_name = sys.argv[1] if len(sys.argv) > 1 else "robot_default"

rospy.init_node(f'publisher_{robot_name}', anonymous=True)
pub = rospy.Publisher('/fleet_status', String, queue_size=10)
rate = rospy.Rate(1)

while not rospy.is_shutdown():
    msg = String()
    msg.data = f"{robot_name} is operational"
    pub.publish(msg)
    rate.sleep()
```

Many-to-one message communications

```
#!/usr/bin/env python3
# fleet_monitor.py - ONE subscriber collecting from all robots

import rospy
from std_msgs.msg import String

def callback(msg):
    rospy.loginfo(f"[FLEET MONITOR] {msg.data}")

rospy.init_node('fleet_monitor')
rospy.Subscriber('/fleet_status', String, callback)
rospy.spin()
```

Many-to-one message communications

```
# Run in separate terminals  
roslaunch my_robot_pkg robot_publisher.py robot_A  
roslaunch my_robot_pkg robot_publisher.py robot_B  
roslaunch my_robot_pkg robot_publisher.py robot_C  
roslaunch my_robot_pkg fleet_monitor.py
```

Many-to-many message communications

```
#!/usr/bin/env python3
# sensor_node.py – Each node both publishes AND subscribes (peer awareness)

import rospy, sys
from std_msgs.msg import String

node_id = sys.argv[1] if len(sys.argv) > 1 else "node_x"

def callback(msg):
    # Ignore our own messages
    if node_id not in msg.data:
        rospy.loginfo(f"[{node_id}] Heard from peer: {msg.data}")
```

Many-to-many message communications

```
rospy.init_node(f'sensor_{node_id}', anonymous=True)
pub = rospy.Publisher('/mesh_network', String, queue_size=10)
rospy.Subscriber('/mesh_network', String, callback)

rate = rospy.Rate(0.5)
while not rospy.is_shutdown():
    msg = String()
    msg.data = f"{node_id} reporting: all systems nominal"
    pub.publish(msg)
    rate.sleep()
```

Many-to-many message communications

```
# Run 3 instances in separate terminals  
roslaunch my_robot_pkg sensor_node.py node_A  
roslaunch my_robot_pkg sensor_node.py node_B  
roslaunch my_robot_pkg sensor_node.py node_C
```

Class activity

Task : "Smart Thermometer"

Create a temperature monitoring system.

- Publisher node generates random temperature values between 15.0°C and 45.0°C and publishes to `/temperature` at 1Hz using `Float32`
- Subscriber node receives and prints the temperature
- If temperature exceeds 35.0°C print a **WARNING**
- If temperature exceeds 40.0°C print a **DANGER — OVERHEAT!**

Deliverables:

- [] `thermometer_publisher.py`
- [] `thermometer_subscriber.py`
- [] Screenshot of terminal output showing warnings



Thank you!

